DnD Campaign Manager API

COMP3011 Web Services CW1

Rory Edmonds

March 14, 2026

Contents

1 Links **2**

2 Introduction **2**

3 Technology Stack **2**

4 System Model **2**

4.1 Characters and Encounter Participants 3

4.2 External Game Data 3

5 System Architecture **3**

5.1 Request Processing Flow 4

5.2 Architecture Layers 4

6 Data Model Design **4**

6.1 Event-Based Logging 5

6.2 Characters and Encounter Participants 5

7 Core Features **5**

8 Testing and Deployment **6**

9 Challenges, Limitations and Future Work **6**

A AI Declaration **7**

B AI Conversation Logs **7**

B.1 Copied Prompt 7

B.2 Screenshots: 9

1 Links

- **GitHub Repository**: https://github.com/BigRorzUni/DND-Campaign-Manager
- **Deployed API**: https://dnd-campaign-manager-production.up.railway.app
- **API Documentation**: https://github.com/BigRorzUni/DND-Campaign-Manager/blob/main/docs/redoc.pdf
- **Presentation Slides**: https://www.canva.com/design/DAHD0-z5FM8/8h_0Q6uMNKkJmtGZZadSlw/edit?utm_content=DAHD0-z5FM8&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton

2 Introduction

This report describes the design and implementation of a RESTful web service for managing Dungeons and Dragons campaigns. This system allows Dungeon Masters (DMs) to record campaigns, sessions, encounters, and gameplay events while providing analytics derived from recorded events.

It is implemented using **FastAPI**, **SQLAlchemy**, and **SQLite** and follows a layered architecture separating routing logic, business logic, repositories, validation schemas, and database models.

The system supports both manual gameplay logging and automated encounter simulation, with both providing flexible analytics but only manual gameplay events contributing to campaign-wide analytics.

3 Technology Stack

I chose Python because I already have web development experience using it. Then, fastAPI was selected because it provides automatic OpenAPI documentation, strong request validation through Pydantic, and high performance compared to other frameworks. I used SQLAlchemy as the ORM to simplify database interaction while maintaining a clear mapping between relational tables and Python models. SQLite was selected for development due to its simplicity and minimal setup requirements, whilst the deployment environment uses PostgreSQL for production. When deploying I used docker to containerise the application, ensuring consistent execution across environments, and deployed to Railway as it was again one that I have experience using and can link to the git repo.

4 System Model

The API models a Dungeons and Dragons campaign using a hierarchical structure:

$$\text{Campaign} \rightarrow \text{Session} \rightarrow \text{Encounter} \rightarrow \text{Event}$$

- **Campaign**: the overall long-running narrative managed by the Dungeon Master.
- **Session**: a single play session within a campaign.
- **Encounter**: a combat interaction that may be simulated or manually logged.
- **Event**: an atomic action such as damage, healing, or spell usage.

This structure allows gameplay to be recorded at a granular level while enabling analytics across encounters, sessions, and entire campaigns.

In addition to the campaign hierarchy, the system models entities that participate in encounters.

4.1 Characters and Encounter Participants

Characters represent persistent entities belonging to a campaign, typically the player characters. Characters store long-term attributes such as class, level, maximum hit points, equipment, and known spells.

During combat, characters are instantiated as *encounter participants*. Participants represent entities that appear in a specific encounter and track temporary state such as current hit points and remaining spell slots.

Separating characters from encounter participants ensures that encounter state does not permanently modify the underlying character record.

Participants may also represent monsters imported from external datasets. This allows encounters to include enemies without adding them to the campaign's persistent character registry.

4.2 External Game Data

The system integrates external datasets describing monsters, spells, and equipment. These datasets allow the API to provide searchable endpoints for importing official DnD content into characters and encounters without requiring the data to be stored directly in the application database.

5 System Architecture

The application follows a layered architecture that separates concerns between HTTP routing, business logic, data access, and persistence. This improves maintainability and testability [5] by isolating different aspects of the system. Avoiding tight coupling between layers allows for easier modifications and extensions in the future.

5.1 Request Processing Flow

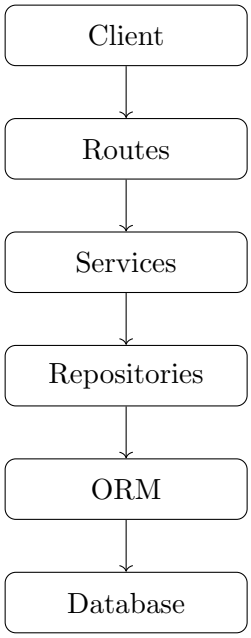


Figure 1: Layered architecture of the Campaign Manager API

When a client sends a request to the API, it is processed through the architecture layers in sequence.

1. Client sends HTTP request
2. Route validates input using Pydantic
3. Service layer executes business logic
4. Repository performs database operations
5. SQLAlchemy interacts with SQLite
6. JSON response returned to client

This layered flow ensures that request validation, business logic, and database access remain clearly separated.

5.2 Architecture Layers

Routes: define HTTP endpoints, validate requests using Pydantic schemas, and return JSON responses. Database sessions are supplied through FastAPI's dependency injection system, ensuring consistent connection handling and allowing dependencies to be overridden during testing.

Services: contain business logic such as encounter simulation, event processing, and analytics calculations.

Repositories: encapsulate database access and isolate query logic from the rest of the application.

Models: define the relational schema using SQLAlchemy ORM.

Schemas: provide request validation and response serialisation.

6 Data Model Design

Campaign data is stored using a relational database schema implemented with SQLAlchemy ORM [2]. A relational model was chosen because the system contains strongly structured relationships between entities such as campaigns, sessions, encounters, and participants[4].

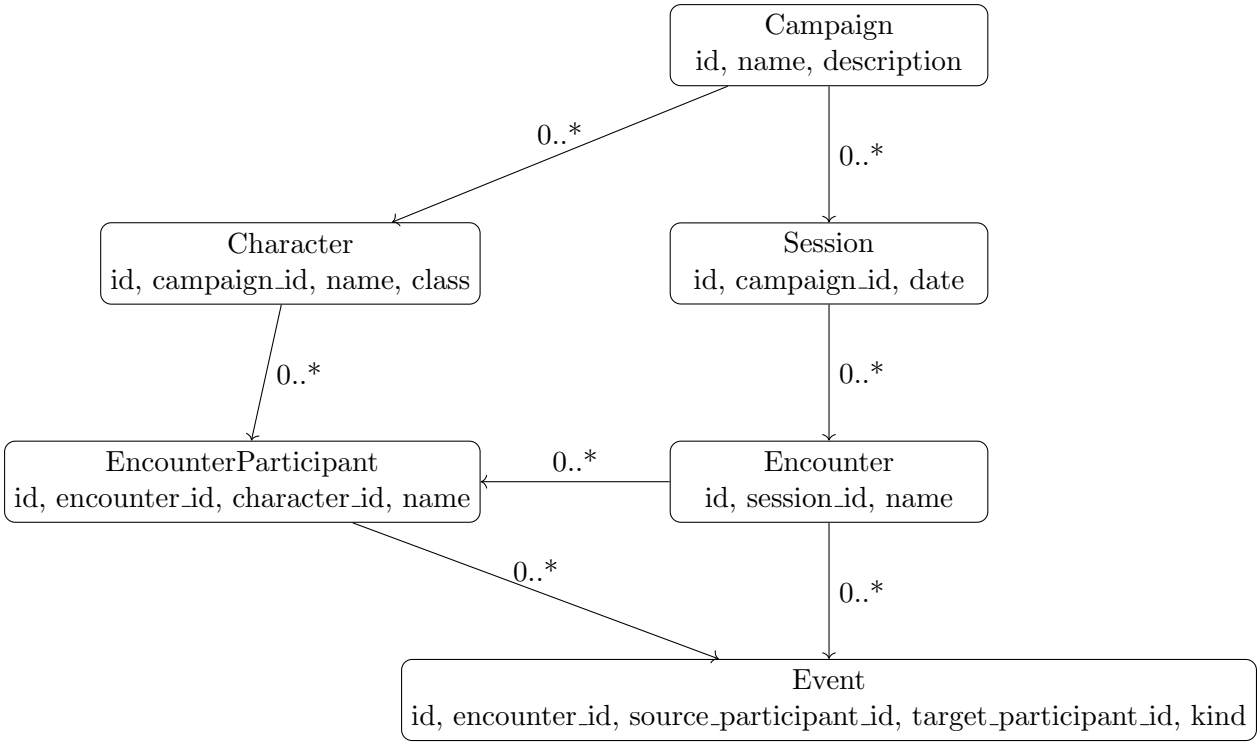


Figure 2: Core Entity Relationships in the Campaign Manager Database

The diagram shows the core entity relationships. Campaigns contain sessions, sessions contain encounters, and encounters contain participants and events. Characters exist at campaign level as persistent entities, while encounter participants represent combatants within a specific encounter and track temporary combat state.

6.1 Event-Based Logging

Gameplay interactions are stored as atomic events, such as damage, healing, or spell usage. Storing events rather than derived statistics allows analytics to be generated dynamically by aggregating event data. It also provides a chronological record of how encounters unfold and allows new analytics features to be added without modifying the schema.

6.2 Characters and Encounter Participants

Characters represent player characters or non-player characters belonging to a campaign. Characters store persistent attributes such as class, level, and equipment. While **Participants** represent entities within an encounter. A participant may be linked to a character or may represent a monster imported from an external dataset. Separating characters from participants allows encounter-specific state such as hit points and spell slots to be tracked independently from persistent character data. It also allows encounters to include temporary entities such as monsters without cluttering the campaign's character registry.

7 Core Features

The system provides several core features that support gameplay tracking and analysis.

Encounter Simulation allows encounters to be automatically resolved. The simulation randomises initiative order, iterates through combat rounds, selects actions for participants, and generates damage or healing events using the same event structure as manual gameplay logs.

Analytics are derived by aggregating event records. These include damage leaderboards, damage taken, healing received, and spell slot usage.

External Datasets are used to provide searchable game content, including spells, equipment, monsters, and character classes. This allows official DnD resources [1]to be imported into characters and encounters without storing the entire dataset in the application database.

8 Testing and Deployment

Automated tests were implemented using `pytest`. Tests use FastAPI's `TestClient` to simulate HTTP requests, allowing the API to be tested in a way that closely mirrors real usage [3].

Each test runs against a temporary SQLite database to ensure isolation from development data and deterministic behaviour. Testing through HTTP requests validates multiple layers of the application simultaneously, including request validation, service logic, repository operations, and database interactions.

Tests verify campaign CRUD operations, session management, encounter lifecycle, participant creation, event logging, and analytics endpoints.

The application was containerised using Docker to ensure a consistent runtime environment. A Dockerfile installs dependencies and runs the FastAPI server using `uvicorn`. Environment variables are used to configure the database connection and API keys, allowing the same container image to be used across development and production. It was then deployed on the Railway cloud platform, which automatically builds and runs the Docker container. In production the service connects to a managed PostgreSQL database provided by Railway.

9 Challenges, Limitations and Future Work

During development several challenges were encountered. One challenge was modelling external dataset information within the character model. Initially, separate character spell and character equipment tables were introduced to represent relationships between characters and imported game data. However, this approach introduced unnecessary schema complexity and made the system more cumbersome to maintain. I simplified by storing references to the external dataset entries rather than duplicating full relational structures. Another challenge was abstracting the complex mechanics of combat into a simplified simulation model. The real game contains many conditional rules and edge cases, so deciding what I would/wouldn't account for in each combat event was very vital.

Several limitations exist in the current implementation, the encounter simulation uses simplified combat rules, authentication and multi-user support are not implemented, external datasets are accessed remotely rather than locally cached so have quite high response time

Future improvements could include richer simulation mechanics, improved analytics visualisation, user authentication, multi-DM campaign support, and improved security.

References

[1] DD 5e SRD API. Dungeons and dragons 5e api. [Online], 2024. [Accessed: 10 Mar 2026]. Available from: https://www.dnd5eapi.co/.

[2] SQLAlchemy Authors. Sqlalchemy orm documentation. [Online], 2024. [Accessed:6 Mar 2026]. Available from: https://docs.sqlalchemy.org.

[3] Sebastián Ramírez. Testing fastapi applications. [Online], 2024. [Accessed: 7 Mar 2026]. Available from: https://fastapi.tiangolo.com/tutorial/testing/.

[4] Abraham Silberschatz, Henry F. Korth, and S. Sudarshan. *Database System Concepts*. McGraw-Hill, 7th edition, 2019.

[5] TestDriven.io. Developing a fastapi crud api. [Online], 2021. [Accessed: 6 Mar 2026]. Available from: https://testdriven.io/blog/fastapi-crud/.

A AI Declaration

Generative AI tools were used during development. It was helpful to review architectural decisions, comment + neaten code, explain how to best use the technology stack, suggest testing strategies, dataset discovery + importing and provide feedback on documentation/report structure and content.

All suggestions were manually reviewed and adapted before being integrated into the project. Final design decisions, implementation, and code structure were determined by me.

B AI Conversation Logs

B.1 Copied Prompt

This response was too long to take effective screenshots so it has been copied

Prompt: Describe a good testing strategy for my system that covers all cases

Response:

```
A good testing strategy for your system should combine unit tests, integration tests, and en

For your DnD Campaign Manager, the strongest approach is:

1. API integration tests as the core

Use pytest with FastAPI's TestClient to send HTTP requests to the application. This is the m
 • request validation
 • routing
 • service logic
 • repository/database behaviour
 • response schemas

These tests should cover the main user workflows, such as:
 • creating, editing, listing, and deleting campaigns
 • creating sessions within campaigns
 • creating encounters within sessions
 • adding party and enemy participants
 • creating and editing events
 • retrieving analytics
```

- running simulations

This gives confidence that the system works as a real API, not just as isolated functions.

2. Temporary isolated test database

Each test or test suite should run against a temporary database, separate from development a

This ensures:
- tests are deterministic
- tests do not corrupt development data
- database state can be reset between tests
- edge cases can be reproduced reliably

A common pattern is to create the schema fresh for tests, override the FastAPI database depe

3. Unit tests for core logic

Some parts of your system are complex enough to justify direct unit tests, especially where
- encounter state recalculation
- simulation decision logic
- event aggregation logic
- spell slot consumption logic
- dataset parsing helpers

These tests are useful for edge cases that would be awkward to drive only through HTTP.

4. Validation and error-case tests

Do not only test the \happy path." You should also test invalid or missing data, for example
- creating entities with missing required fields
- invalid IDs
- deleting non-existent resources
- invalid spell or equipment imports
- running simulations on malformed encounters
- invalid action references

These tests show that the API fails safely and returns appropriate HTTP status codes.

5. Analytics correctness tests

Because analytics are a major feature, test not just whether endpoints return 200 OK, but wh
- total damage dealt matches recorded events
- healing totals are correct
- spell slot usage is counted correctly
- encounter review metrics match the event log

This is especially important because your design depends on aggregating events rather than s

6. Simulation-specific tests

Since simulation is one of your distinguishing features, include tests for:

- simulated encounters generating events
- winner and round count being updated
- participant HP changing correctly
- spell slots decreasing where appropriate
- simulated encounters locking manual event editing if that is part of your design

This demonstrates that the simulator is not just present, but testable.

7. External dataset tests

Because you depend on external monsters, spells, and equipment, test that:

- dataset lookup endpoints return expected structures
- imports correctly populate internal entities
- failures are handled gracefully when an external item is missing

In a fuller production system, these would ideally be mocked to avoid network dependence dur

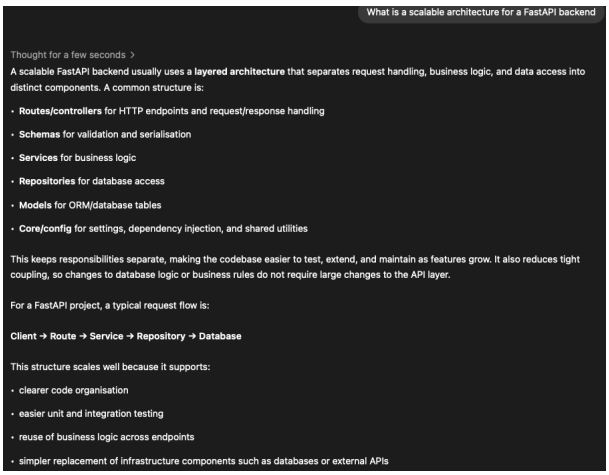
8. Regression tests

Whenever you fix a bug, add a test for it. This is especially useful for:

- participant state tracking
- simulation fallbacks
- spell slot visibility logic
- imported monster behaviour
- action resolution bugs

This prevents old problems from reappearing.

B.2 Screenshots:



How should a system model hierarchical gameplay data such as campaigns, sessions, encounters, and combat?

A good way to model hierarchical gameplay data is to represent it as a **relational hierarchy** where each level owns the level below it. For example:

- a **campaign** contains many **sessions**
- a **session** contains many **encounters**
- an **encounter** contains many **events** or combat actions

This structure reflects how gameplay is organised in practice and makes it easy to query data at different levels, such as all encounters in a campaign or all events in a session.

Using foreign keys between these entities helps maintain consistency and ensures that related records remain properly connected. It also supports analytics at multiple levels, since data can be aggregated per encounter, per session, or across the whole campaign.

If combatants need temporary state during an encounter, it is also useful to model them separately from the persistent campaign-level characters. This allows combat-specific values such as current hit points or remaining spell slots to be tracked without permanently altering the underlying character record.

how should player combat data be tracked in encounters without permanently modifying the overall character records

Player combat data should be tracked using a **separate encounter-specific entity** rather than storing changing combat values directly on the main character record.

A good approach is:

- keep **characters** as persistent campaign-level records containing long-term attributes such as name, class, level, max hit points, equipment, and known spells
- create an **encounter participant** record whenever that character enters an encounter
- store temporary combat state on the participant, such as:
  - current hit points
  - remaining spell slots
  - temporary notes or conditions
  - encounter-specific combat role or status

This allows combat to be simulated or logged without overwriting the character's long-term data. Once the encounter ends, the participant record represents what happened during that fight, while the original character record remains the canonical persistent version.

This design also makes analytics easier, because combat outcomes can be analysed per encounter without mixing temporary and permanent character state. It also allows non-persistent enemies, such as monsters, to be represented in the same participant structure.

should i record statistics directly into the database or as 'events' that can be later analysed

In most cases, it is better to record gameplay actions as **events** and derive statistics from them later, rather than storing only precomputed totals directly in the database.

An event-based approach gives you a record of what actually happened, for example:

- damage dealt
- healing received
- spell usage
- other encounter actions

From that event history, statistics such as total damage, healing, or spell slot usage can be calculated when needed.

This has several advantages:

- it preserves a chronological record of gameplay
- it makes analytics more flexible, since new statistics can be added later without changing the schema
- it reduces the risk of stored totals becoming inconsistent with the underlying actions
- it supports both manual logging and simulation using the same representation

Directly storing aggregate statistics can be useful as a cache or optimisation, but it is usually less flexible if you want to support richer analysis later.

how should i import the spell, monster, equipment and character classes without storin g the whole thing in a database?

A good approach is to treat spells, monsters, equipment, and character classes as **external reference data** rather than as core application data stored permanently in your own database.

Instead of copying the full dataset into local tables, the system can:

- query the external API when a user searches for a spell, monster, or item
- return a simplified summary of the external data to the client
- store only the **small identifying reference** needed internally, such as an index or ID
- optionally cache selected responses temporarily if performance becomes an issue

For example:

- a character can store a list of **spell indices**
- an encounter participant can store a **monster index**
- equipment can be linked by its external **equipment index**
- class selection can use the class name or class index returned by the external dataset

When more detail is needed, the application can fetch it from the external dataset service on demand. This avoids duplicating large amounts of external content, keeps the local schema smaller, and makes it easier to benefit from updates to the source dataset.